

Resolución: Problema de Suma de Subconjuntos mediante Programación Dinámica

Este documento presenta la formalización matemática, el diseño de algoritmos iterativos y la reconstrucción de soluciones para el problema de decidir si existe un subconjunto de N objetos cuyas longitudes sumen exactamente una longitud objetivo L .

1. Ecuación en Recurrencias

Definimos la función booleana $S[i, w]$ que representa si es posible obtener una suma de longitudes exactamente igual a w ($0 \leq w \leq L$) utilizando únicamente un subconjunto de los primeros i objetos ($0 \leq i \leq N$).

Casos Base

1. **Suma de longitud cero ($w = 0$):** Siempre es posible obtener una suma de 0 eligiendo el conjunto vacío (sin objetos), independientemente de cuántos objetos tengamos disponibles:

$$S[i, 0] = \text{Verdadero} \quad \forall i \in \{0, 1, \dots, N\}$$

2. **Sin objetos disponibles ($i = 0$) para una suma positiva ($w > 0$):** Es imposible obtener una suma estrictamente mayor que cero si no disponemos de ningún objeto:

$$S[0, w] = \text{Falso} \quad \forall w \in \{1, 2, \dots, L\}$$

Caso General (Paso de Recurrencia)

Para calcular $S[i, w]$ con $1 \leq i \leq N$ y $1 \leq w \leq L$, analizamos si decidimos incluir o no el i -ésimo objeto de longitud l_i :

- **Si el objeto actual es demasiado largo ($l_i > w$):** No podemos incluirlo en el subconjunto. Por tanto, dependemos exclusivamente de las decisiones con los objetos anteriores:

$$S[i, w] = S[i - 1, w]$$

- **Si el objeto actual cabe ($l_i \leq w$):** Tenemos dos opciones válidas:
 - *No incluirlo:* El valor se mantiene igual a la solución sin él ($S[i - 1, w]$).
 - *Incluirlo:* Buscamos si es posible formar la suma restante ($w - l_i$) con los primeros $i - 1$ objetos ($S[i - 1, w - l_i]$).

Uniendo ambas opciones lógicas obtenemos:

$$S[i, w] = S[i - 1, w] \vee S[i - 1, w - l_i]$$

2. Algoritmo Iterativo con Espacio Optimizado $O(L)$

Para optimizar el espacio a $O(L)$, observamos que para calcular la fila i de la tabla solo necesitamos los datos de la fila anterior $i - 1$.

Podemos utilizar un único vector unidimensional de tamaño $L + 1$. Para evitar que los valores actualizados de la iteración actual interfieran con los cálculos de la misma fila, **recorremos el presupuesto w en sentido descendente** (desde L hasta l_i).

Algoritmo en Pseudocódigo

```

Funcion ExisteSubconjuntoL(N, L, l)
Inicio
  // Crear una tabla unidimensional de tamaño L + 1
  DP <- Vector de tamaño L + 1 inicializado en Falso

  // Caso base: una suma de 0 siempre es alcanzable
  DP[0] <- Verdadero

  // Llenar el vector iterativamente para cada objeto
  Para i <- 1 Hasta N Con Paso 1 Hacer
    // Recorrido inverso para usar únicamente valores de la iteración anterior
    Para w <- L Hasta l[i] Con Paso -1 Hacer
      DP[w] <- DP[w] o DP[w - l[i]]
    FinPara
  FinPara

  Retornar DP[L]
FinFuncion

```

Análisis de Costes

- **Complejidad Temporal:** $O(N \cdot L)$. El algoritmo utiliza dos bucles anidados. El bucle externo realiza N iteraciones y el interno realiza como máximo L iteraciones de coste constante $O(1)$.
- **Complejidad Espacial:** $O(L)$ debido al almacenamiento de un único vector DP de tamaño $L + 1$.

3. Modificación para Obtener el Subconjunto de Objetos

Si además de saber si existe una solución deseamos **reconstruir el subconjunto**, la optimización de espacio a un vector de una sola dimensión no es suficiente, ya que perdemos el histórico de las decisiones previas.

Para recuperar el subconjunto, debemos almacenar la matriz bidimensional completa M de dimensiones $(N + 1) \times (L + 1)$ y luego realizar una reconstrucción hacia atrás (*backtracking*).

Algoritmo en Pseudocódigo

```
Funcion ObtenerSubconjuntoExacto(N, L, l)
Inicio
  // 1. Crear e inicializar la matriz bidimensional completa
  M <- Matriz de dimensiones (N + 1) x (L + 1) inicializada en Falso

  // Casos base
  Para i <- 0 Hasta N Con Paso 1 Hacer
    M[i, 0] <- Verdadero
  FinPara

  // 2. Rellenar la matriz de programación dinámica
  Para i <- 1 Hasta N Con Paso 1 Hacer
    Para w <- 1 Hasta L Con Paso 1 Hacer
      Si l[i] > w Entonces
        M[i, w] <- M[i - 1, w]
      Sino
        M[i, w] <- M[i - 1, w] o M[i - 1, w - l[i]]
      FinSi
    FinPara
  FinPara

  // Si no existe ninguna combinación que sume L, devolvemos una lista vacía
  Si M[N, L] = Falso Entonces
    Escribir("No existe ningún subconjunto con la suma exacta.")
    Retornar Lista vacía
  FinSi

  // 3. RECONSTRUCCIÓN: Obtener los elementos seleccionados hacia atrás
  objetos_seleccionados <- Lista vacía
  w_actual <- L

  Para i <- N Hasta 1 Con Paso -1 Hacer
    // Si el estado de la celda actual es verdadero, pero el de la fila anter
    // significa que el objeto i fue seleccionado obligatoriamente para logro
    Si M[i, w_actual] = Verdadero y M[i - 1, w_actual] = Falso Entonces
      Añadir i a objetos_seleccionados
      w_actual <- w_actual - l[i] // Restamos la longitud aportada por el c
    FinSi
  FinPara

  Retornar objetos_seleccionados
FinFuncion
```